

Overview of Statistical Learning and Modeling – 36-600

Lab 11R: Numerical Optimization
Due Friday, November 14, 11:59pm

Trishla Nair

Data

Let's generate the same data that we used to illustrate nonlinear regression:

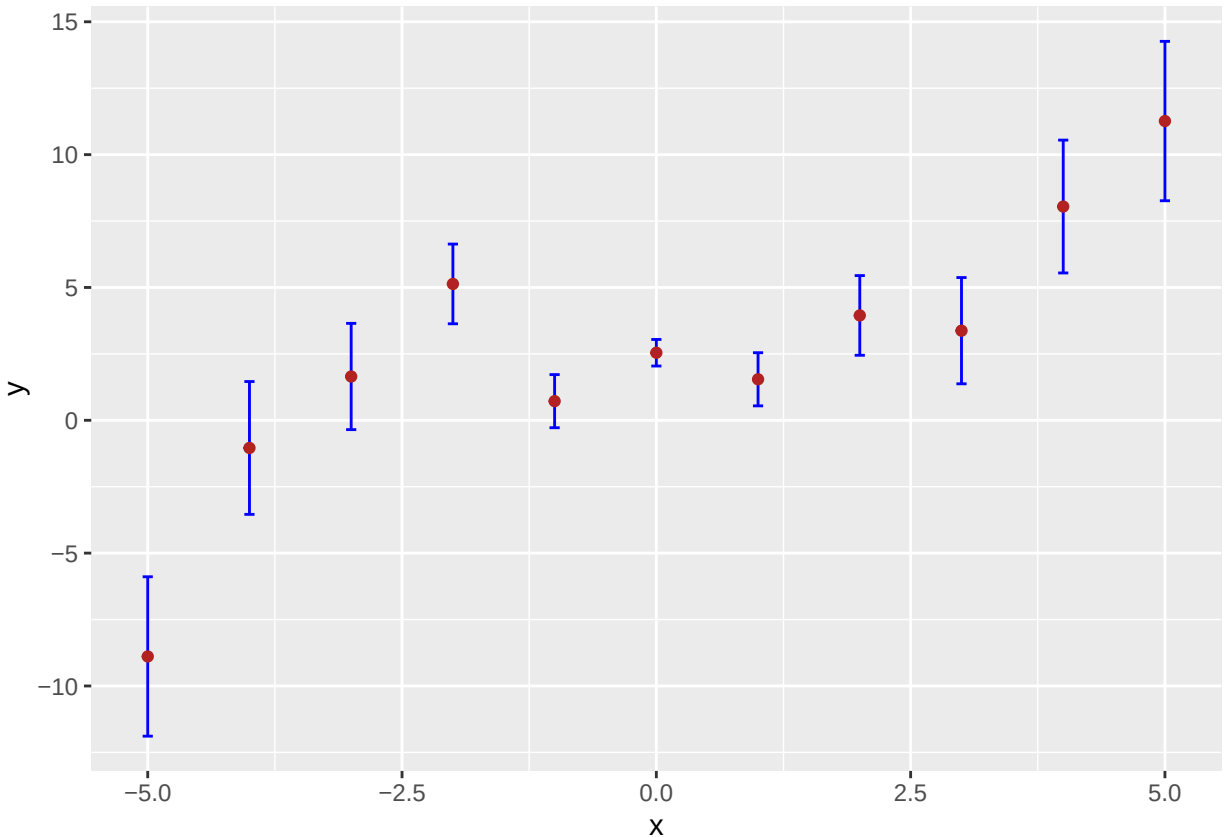
```
set.seed(555)
x <- -5:5
y <- 0.1*x^3 - 0.5*x + 2.1 + rnorm(length(x), mean = 0, sd = 0.5*(1 + abs(x)))
e <- 0.5*(1 + abs(x))

df <- data.frame("x" = x, "y" = y, "e" = e)

suppressMessages(library(tidyverse))
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
ggplot(data = df, mapping = aes(x = x, y = y)) +
  geom_errorbar(aes(ymin = y-e, ymax = y+e), width = 0.1, color = "blue") +
  geom_point(color = "firebrick")
```



Questions

Question 1

Assume that you know that a cubic polynomial is the correct function to use here, but you don't know what the true parameters are. (That means you should include the x^2 term in your curve-fitting analysis! Of course, it's true that you could just do polynomial regression here, like you did in the last lab, but let's ahead and try out our shiny new numerical optimization techniques.) Code an optimizer that will allow you to estimate the four coefficient terms of a cubic polynomial. Try not to let the true cubic polynomial coefficients above influence your initial guesses. Show the coefficients and plot your result. Make sure your plot looks good before you move on to Question 2; with four terms, it can be relatively easy to find a locally optimal result that is very much **not** the globally optimal result, i.e., it can be relatively easy to find a local minimum in χ^2 that is not the global minimum. It's pretty easy to identify when this is the case when you plot and *look at* the functions - always plot them!

```
chi_sq <- function(params) {
  a <- params[1]
  b <- params[2]
  c <- params[3]
  d <- params[4]
  y_pred <- a + b*df$x + c*df$x^2 + d*df$x^3
  chi2 <- sum(((df$y - y_pred)/df$e)^2)
  return(chi2)
}
```

```

# Random but reasonable starting guesses
set.seed(123)
start_vals <- runif(4, -1, 1)

# Run optimization
fit <- optim(par = start_vals, fn = chi_sq)

# Display coefficients
fit$par

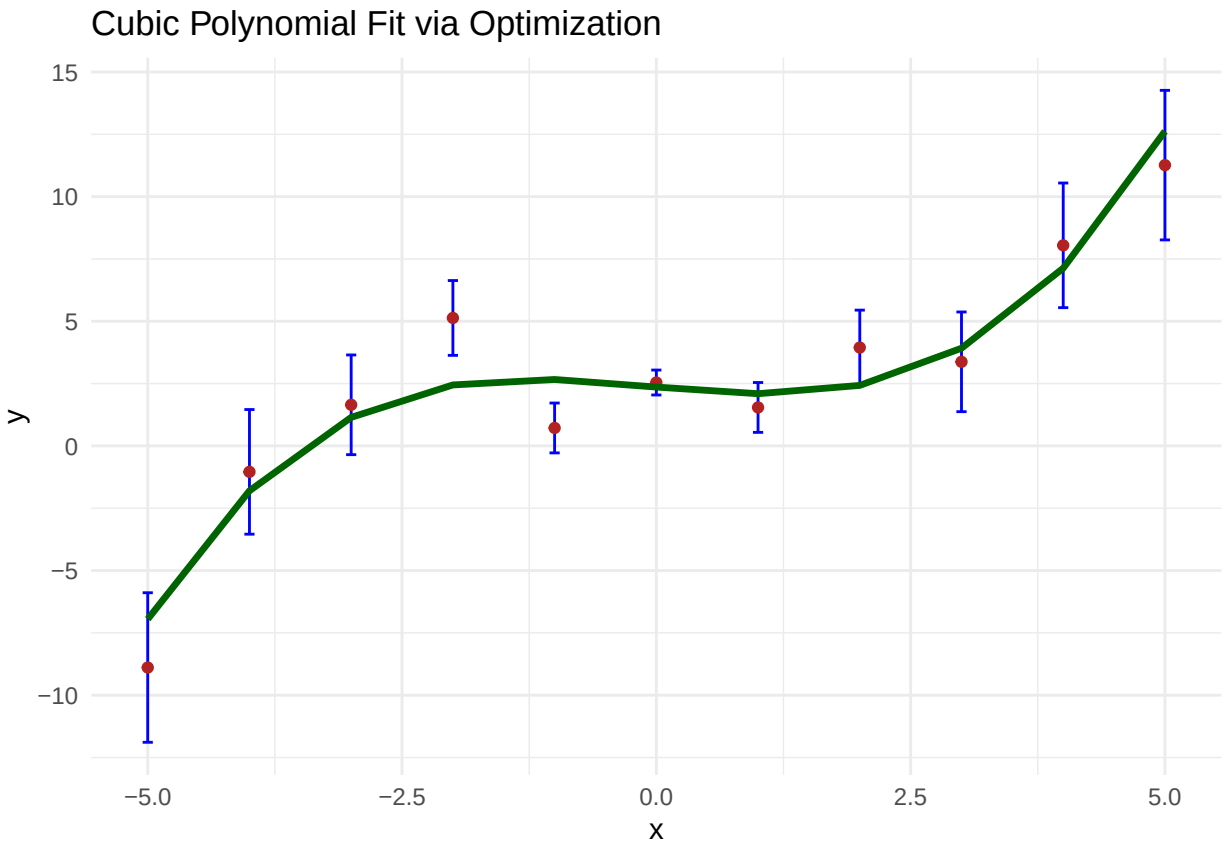
## [1] 2.36037768 -0.37853014 0.01900517 0.09341833

# Plot results
df$y_fit <- fit$par[1] + fit$par[2]*df$x + fit$par[3]*df$x^2 + fit$par[4]*df$x^3

suppressMessages(library(tidyverse))
ggplot(df, aes(x = x)) +
  geom_errorbar(aes(ymin = y - e, ymax = y + e), color = "blue", width = 0.1) +
  geom_point(aes(y = y), color = "firebrick") +
  geom_line(aes(y = y_fit), color = "darkgreen", size = 1.2) +
  theme_minimal() +
  ggtitle("Cubic Polynomial Fit via Optimization")

## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

```



Question 2

Take the minimum χ^2 value that you found in Question 1 (it's in the output from `optim()`, even if you didn't explicitly display it) and perform a χ^2 goodness-of-fit test. Recall that the null hypothesis is that the model is an acceptable one, i.e., that it plausibly replicates the data-generating process. Do you reject the null hypothesis, or fail to reject the null? (Also recall that the number of degrees of freedom is $n - p$, where n is the length of x and p is the number of coefficients in the cubic polynomial.)

```
chi2_min <- fit$value
n <- nrow(df)
p <- length(fit$par)
dfree <- n - p

p_value <- 1 - pchisq(chi2_min, dfree)

cat("Minimum  $\chi^2$  =", round(chi2_min, 3), "\n")
```

```
## Minimum  $\chi^2$  = 9.44
```

```
cat("Degrees of freedom =", dfree, "\n")
```

```
## Degrees of freedom = 7
```

```
cat("p-value =", round(p_value, 4), "\n")
```

```
## p-value = 0.2226
```

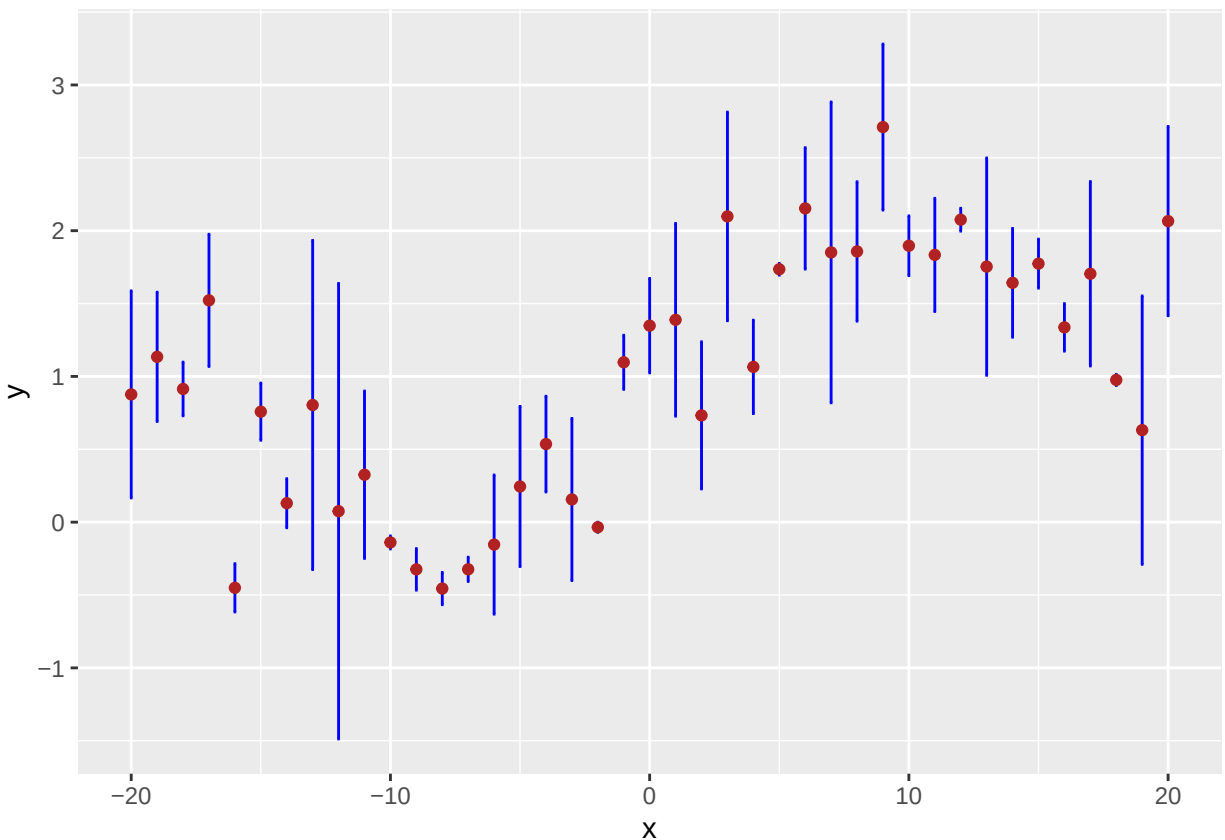
Since the p-value is much larger than 0.05, we fail to reject the null hypothesis and the model is plausible.

Data Part II

Now let's say we have a dataset that looks like this:

```
df <- read.csv("https://raw.githubusercontent.com/FSKoerner/F25-36600-data/refs/heads/main/optim_data.csv")

suppressMessages(library(tidyverse))
ggplot(data = df, mapping = aes(x = x, y = y)) +
  geom_errorbar(aes(ymin = y-e, ymax = y+e), width = 0.1, color = "blue") +
  geom_point(color = "firebrick")
```



Question 3

This will be a bit more free-form. Determine a model that might generate the observed data, optimize its parameters, and plot the result. Not sure where to start? Well, that's okay; it's a *very* common feeling in real-life data analysis.

```

chi_sq_exp <- function(params) {
  a <- params[1]
  b <- params[2]
  c <- params[3]

  y_pred <- a * exp(-b * df$x) + c
  chi2 <- sum(((df$y - y_pred)/df$e)^2)
  return(chi2)
}
start_vals <- c(a = 5, b = 0.5, c = 1)

fit_exp <- optim(par = start_vals, fn = chi_sq_exp)
fit_exp$par

```

```

##           a           b           c
## -0.09480368 -0.02536135  0.84958573

```

```

df$y_fit <- fit_exp$par[1] * exp(-fit_exp$par[2]*df$x) + fit_exp$par[3]

ggplot(df, aes(x = x)) +
  geom_errorbar(aes(ymin = y - e, ymax = y + e), color = "blue", width = 0.1) +
  geom_point(aes(y = y), color = "firebrick") +
  geom_line(aes(y = y_fit), color = "darkgreen", size = 1.2) +
  theme_minimal() +
  ggtitle("Fitted Exponential Model via Optimization")

```

Fitted Exponential Model via Optimization

